UNIVERSIDAD TÉCNICA DE MANABÍ
TECNOLOGÍAS DE LA INFORMACIÓN
DESARROLLO DE SISTEMAS INFORMÁTICOS

ACTIVIDAD #1

ESTUDIO DE CASO
SISTEMA DE CONTROL DE RESCATE Y ADOPCIÓN ANIMAL

Estudiante: STEFY VALERIA PALACIOS DE LA CRUZ

Docente: _____

Nivel: Quinto

Paralelo: A

Periodo académico: Septiembre 2026 – Enero 2027

Fecha de entrega: 17 de octubre de 2026

Enlace al repositorio GitHub: https://github.com/StefyValeriaPalacios/sistema-rescate-adopcion-animal

Contenido

1. Introducción

2. Objetivos

3. Alcance y descripción del problema

4. Actores del sistema

5. Requerimientos

6. Diagrama de casos de uso

7. Diagrama de clases UML

8. Diagramas de secuencia y arquitectura

9. Patrones de diseño aplicados

12. Estructura recomendada del repositorio

13. Pruebas conceptuales y validaciones

14. Conclusiones

15. Referencias

1. Introducción

El rescate y la adopción de animales requieren una gestión organizada que permita registrar reportes, coordinar voluntarios, documentar la atención veterinaria y dar seguimiento a las solicitudes de adopción. Cuando la información se mantiene en registros dispersos, puede ser difícil conocer el estado real de cada caso y evitar duplicidad de datos.

Este proyecto propone la arquitectura lógica de un sistema de Control de Rescate y Adopción Animal. La solución centraliza los datos de ciudadanos reportantes, animales, casos de rescate, voluntarios, veterinarios y solicitudes de adopción. El diseño se apoya en UML, patrones de diseño y control de versiones con GitHub y Gitflow.

2. Objetivos

Objetivo general

Diseñar la arquitectura lógica de un sistema de gestión de rescate y adopción animal aplicando modelos UML, patrones de diseño y el flujo de trabajo Gitflow para favorecer una solución organizada, escalable y mantenible.

Objetivos específicos:

- Representar las interacciones y la estructura del software mediante diagramas UML.
- Justificar la aplicación de al menos tres patrones de diseño adecuados al dominio del problema.
- Establecer un flujo de ramas y commits en GitHub para documentar el desarrollo del diseño.

3. Alcance y descripción del problema

El Sistema de Control de Rescate y Adopción Animal es una propuesta de software diseñada para organizar y facilitar la gestión de los casos de animales que necesitan ayuda. Su finalidad es centralizar la información de los reportes, los animales rescatados, los voluntarios, los veterinarios y las solicitudes de adopción, permitiendo un seguimiento ordenado de cada caso.

El sistema permitirá que los ciudadanos registren reportes sobre animales en situación de riesgo, incluyendo información como la ubicación, la especie y una descripción del caso. Posteriormente, el administrador del refugio podrá revisar los reportes, asignar responsables y supervisar el progreso de las actividades de rescate.

El alcance de esta propuesta comprende el análisis de requisitos, la identificación de actores, el diseño de diagramas UML, la aplicación de patrones de diseño y principios SOLID, y la organización del proyecto mediante GitHub y Gitflow. El diseño busca facilitar el mantenimiento del software, permitir la incorporación de nuevas funcionalidades y mejorar la organización de la información relacionada con el rescate y la adopción de animales.

5. Actores del Sistema

Ciudadano/Reportante: comunica la existencia de un animal en riesgo, aporta ubicación y descripción y consulta el estado del reporte cuando esté habilitado.

Voluntario/Rescatista: recibe asignaciones, registra la atención en campo y actualiza el progreso del rescate.

Veterinario: registra evaluaciones, tratamientos y recomendaciones sobre el estado de salud.

Administrador del Refugio: administra usuarios, clasifica casos, asigna responsables y supervisa reportes.

Responsable de adopciones: revisa solicitudes, registra la decisión y documenta la entrega. Este rol puede asignarse a un usuario autorizado del refugio.

Requerimientos funcionales

RF01. Registro de casos de rescate: El sistema permitirá registrar nuevos casos de animales que se encuentren en situación de riesgo, incluyendo la descripción del caso, la ubicación, la fecha del reporte y los datos del reportante.
RF02. Clasificación de animales: El sistema permitirá categorizar a los animales según su especie, como perros, gatos y fauna silvestre, para facilitar su identificación y gestión.
RF03. Gestión de información de animales: El sistema permitirá registrar y consultar la información de cada animal rescatado, incluyendo sus características, estado de salud y situación actual.
RF04. Asignación de responsables: El administrador del refugio podrá asignar voluntarios o veterinarios para atender los casos de rescate, de acuerdo con las necesidades de cada animal.
RF05. Actualización del estado del caso: El sistema permitirá actualizar el estado de los casos de rescate, utilizando estados como reportado, rescatado, en tratamiento y disponible para adopción.
RF06. Registro de atención veterinaria: El sistema permitirá registrar las evaluaciones, los tratamientos y las recomendaciones realizadas por el personal veterinario.
RF07. Gestión de solicitudes de adopción: El sistema permitirá registrar las solicitudes de adopción, consultar su estado y documentar la aprobación o el rechazo por parte del responsable autorizado.
RF08. Notificaciones automáticas: El sistema notificará a los voluntarios o responsables correspondientes cuando se registre un nuevo caso de rescate o se produzcan cambios importantes en su estado.

Requerimientos no funcionales

RNF01. Mantenibilidad: El sistema deberá contar con una arquitectura organizada y modular que facilite la

corrección de errores, la actualización de funcionalidades y la incorporación de mejoras.
RNF02. Escalabilidad: La arquitectura deberá permitir incorporar nuevas funcionalidades, especies animales, usuarios y procesos sin necesidad de rediseñar completamente el sistema.
RNF03. Seguridad: El sistema deberá controlar el acceso a las funcionalidades según los permisos de cada usuario y proteger la información personal de los ciudadanos y responsables.
RNF04. Trazabilidad de versiones: El proyecto utilizará GitHub y el modelo de trabajo Gitflow para registrar los cambios, organizar las ramas y mantener un historial verificable del desarrollo.
RNF05. Reutilización: El diseño deberá promover la reutilización de componentes y clases mediante patrones de diseño y principios SOLID, reduciendo la duplicación de código.
RNF06. Integridad de la información: El sistema deberá mantener la coherencia de los datos de animales, reportes, asignaciones y solicitudes de adopción, evitando registros inconsistentes.
RNF07. Usabilidad: La interfaz deberá presentar opciones claras y una organización comprensible para facilitar el uso por parte de administradores, voluntarios y demás usuarios autorizados.

8. Diagrama de casos de uso UML

El diagrama identifica las operaciones principales del sistema y qué actor inicia cada interacción. El Ciudadano/Reportante registra un reporte y consulta su estado; el Voluntario gestiona las actividades de rescate; el Veterinario registra la atención médica; el Administrador administra usuarios, asignaciones y reportes; y el Responsable de adopciones evalúa solicitudes y registra decisiones.

Sistema de control de rescate y adopcion animal

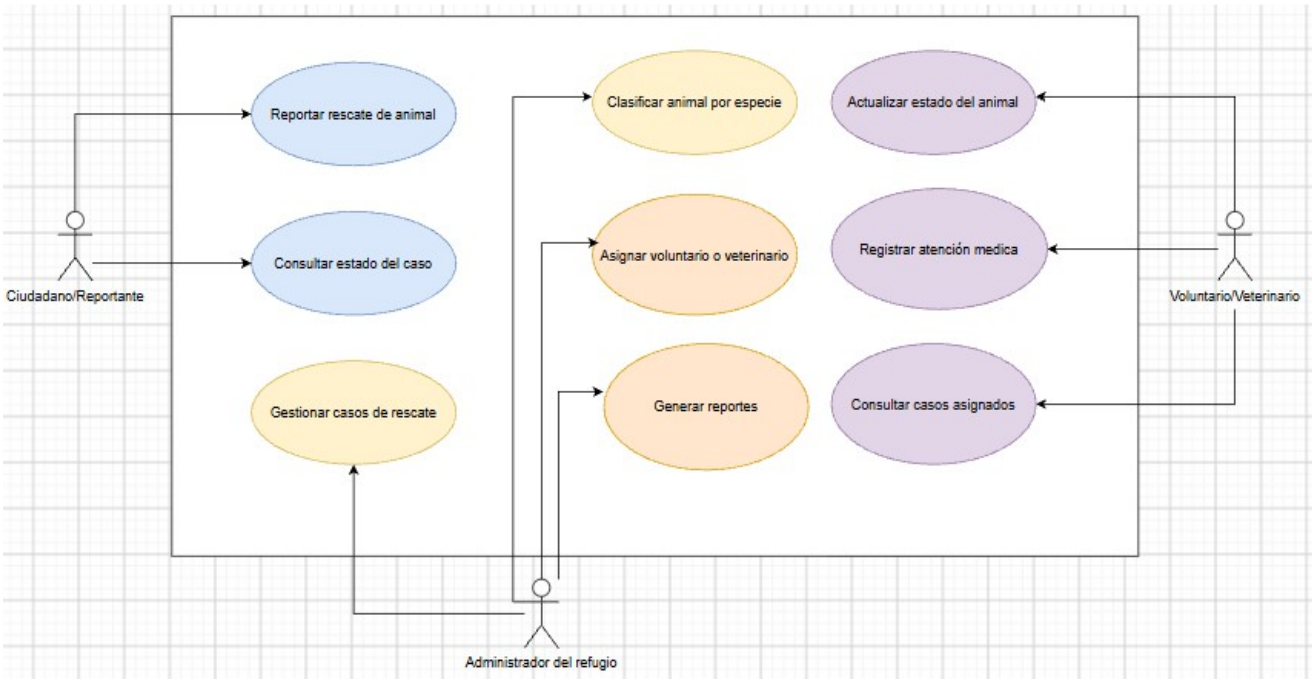
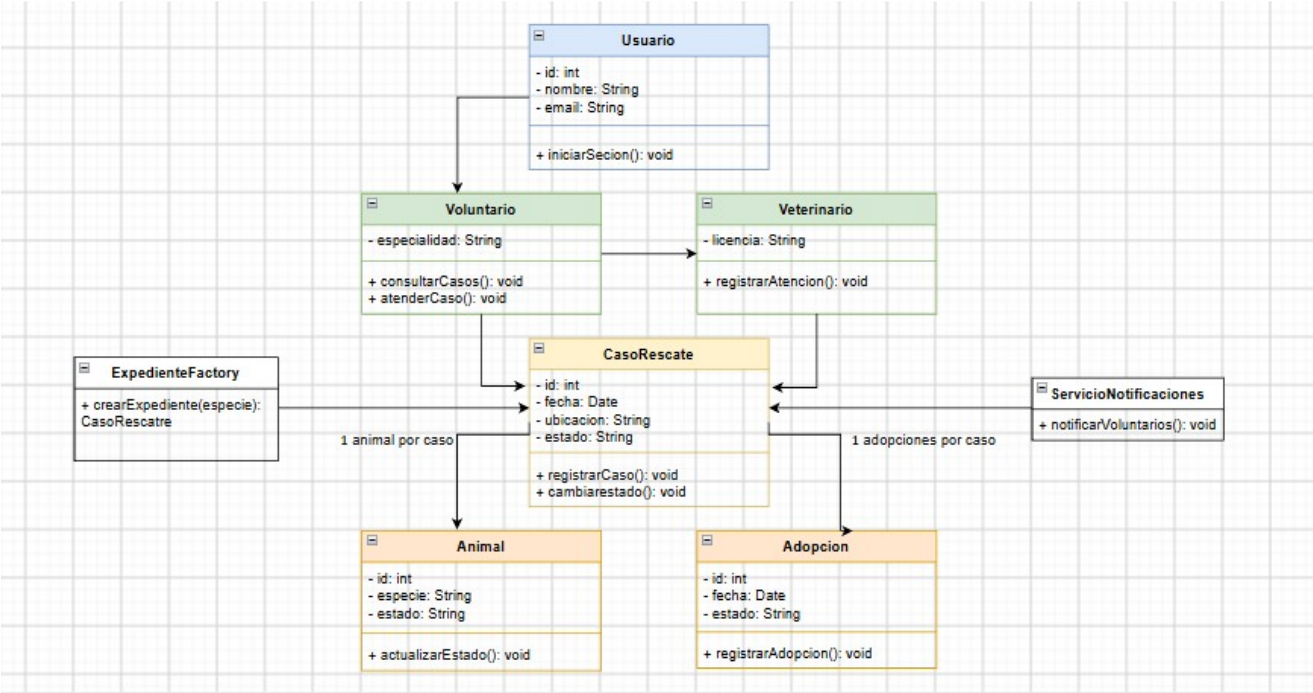


Diagrama de clases UML

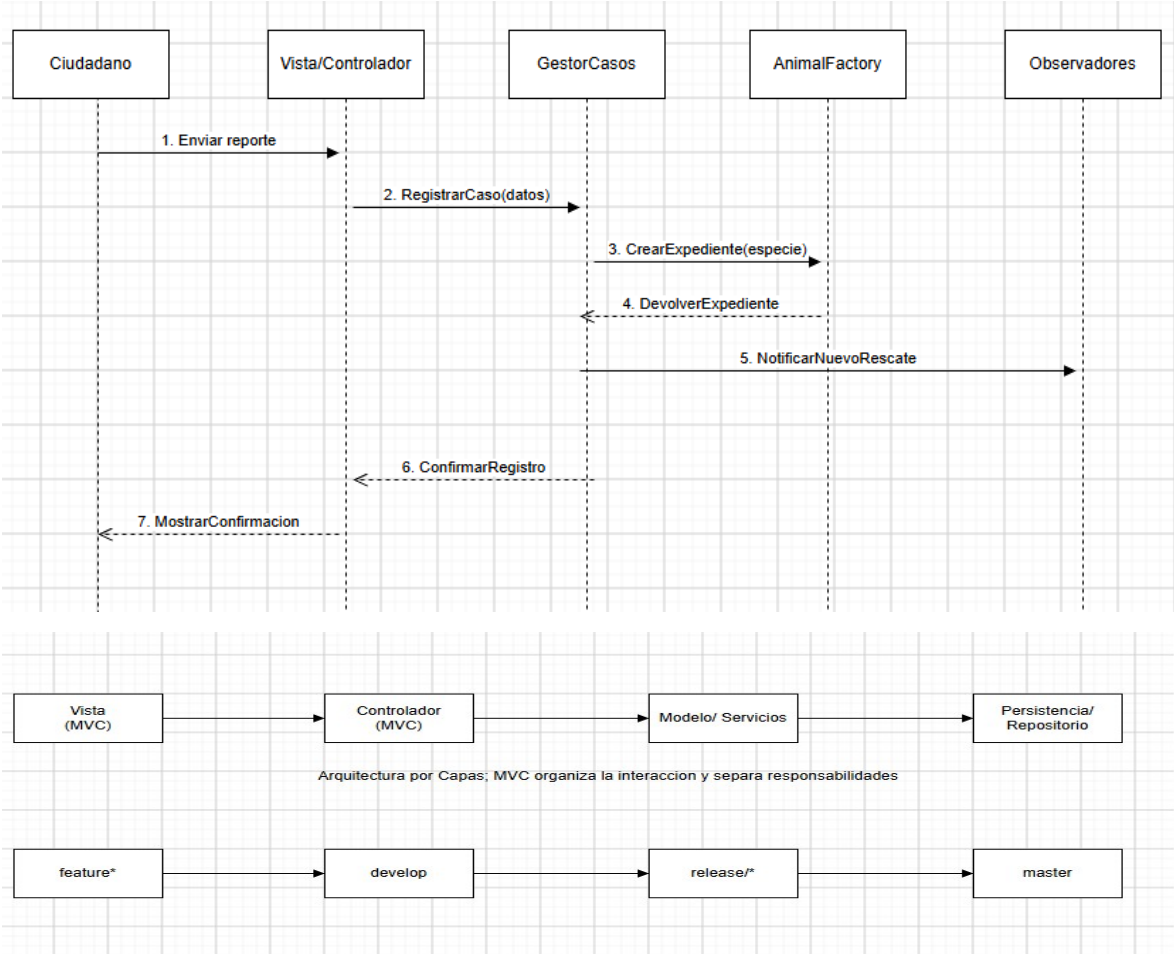
Usuario contiene los datos comunes de identificación y acceso. Ciudadano, Voluntario, Veterinario y Administrador pueden especializar al usuario cuando existan diferencias de comportamiento y permisos. Animal representa al animal rescatado y conserva su especie y estado. ReporteRescate almacena la información del incidente. EvaluacionMedica registra el historial de salud. SolicitudAdopcion relaciona a un adoptante con un animal y registra la decisión.



Principios SOLID: (S) cada clase tiene una responsabilidad clara; (O) pueden añadirse nuevos tipos de expediente implementando una interfaz; (L) los expedientes concretos deben poder sustituir al tipo base sin romper el comportamiento; (I) las interfaces deben ser pequeñas y específicas; (D) los servicios deben depender de abstracciones, por ejemplo una interfaz de notificación, y no de una clase concreta de correo.

8. Diagramas de secuencia y arquitectura

El diagrama de secuencia describe el escenario de registrar un nuevo rescate: el ciudadano envía un reporte, la vista/controlador valida los datos, GestorCasos registra el caso, AnimalFactory crea el expediente según la especie y el mecanismo Observer notifica a los componentes suscritos. Finalmente, el sistema confirma el registro.



La arquitectura propuesta utiliza MVC. La Vista presenta formularios y resultados; el Controlador recibe solicitudes y coordina validaciones; el Modelo contiene las entidades y reglas del negocio; y la capa de persistencia se encarga de guardar y consultar datos. Los patrones se ubican en la capa de servicios para mantener la interfaz separada de las reglas de negocio.

9. Patrones de diseño aplicados

9.1 Singleton (creacional): se utiliza en GestorCasos si el sistema necesita una única instancia de coordinación dentro de la aplicación. Evita crear múltiples gestores con estados inconsistentes. La conexión a base de datos debe gestionarse preferentemente mediante un pool o un servicio de persistencia; Singleton no significa abrir una única conexión global para todas las operaciones.

9.2 Observer (comportamiento): permite que varios componentes se suscriban a eventos de rescate. Al registrarse un nuevo caso, el sistema emite un evento y los observadores, como el módulo de notificaciones o el tablero del refugio, reaccionan sin que GestorCasos dependa de sus clases concretas. Las notificaciones fallidas deben registrarse y poder reintentarse.

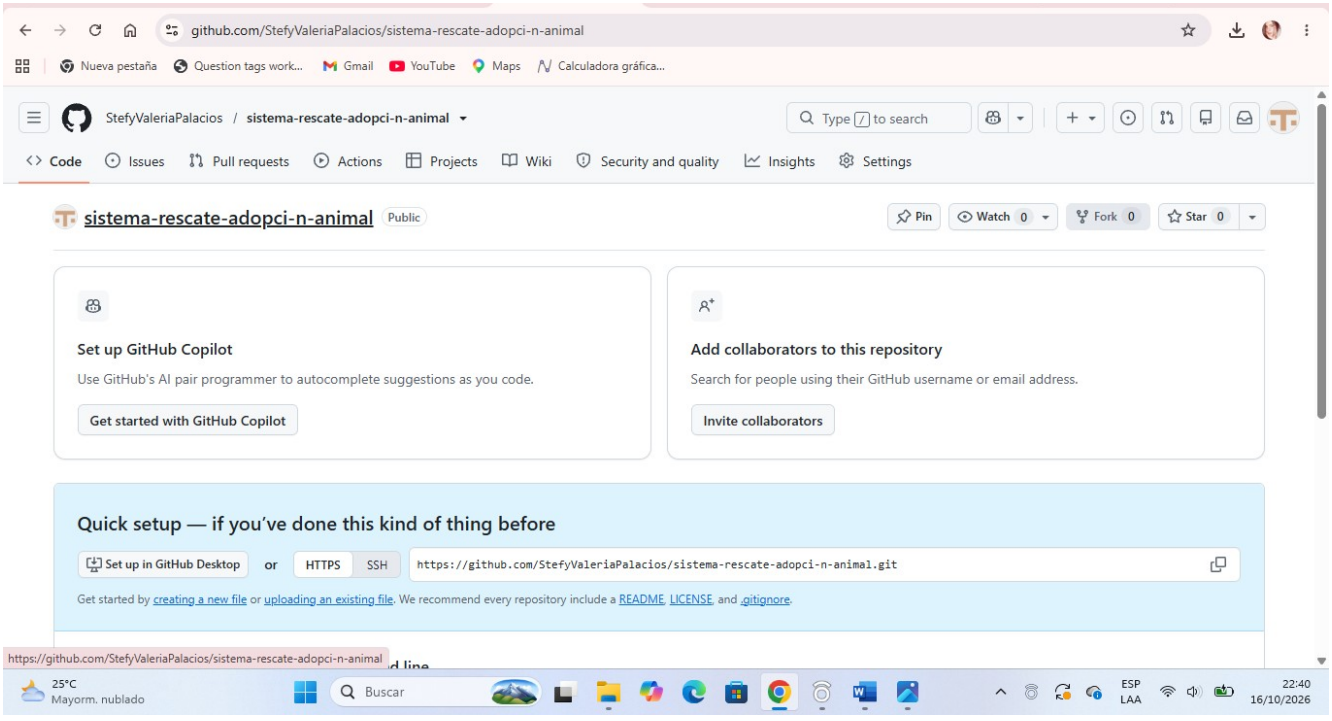
9.3 Factory Method (creacional): encapsula la creación de expedientes según la especie. AnimalFactory puede producir ExpedientePerro, ExpedienteGato o ExpedienteFaunaSilvestre. Así, el código que registra el caso no necesita conocer todos los detalles de construcción de cada expediente. Para agregar una especie, se incorpora una implementación nueva y se actualiza el mecanismo de selección.

9.4 MVC (arquitectónico): separa Vista, Controlador y Modelo. La vista no debe contener reglas del negocio; el controlador coordina la petición y el modelo representa los datos y comportamientos. MVC complementa los otros patrones y mejora la organización general.

Los tres patrones mínimos solicitados quedan cubiertos por Singleton, Observer y Factory Method; MVC se incluye como patrón arquitectónico complementario.

12. Estructura del repositorio

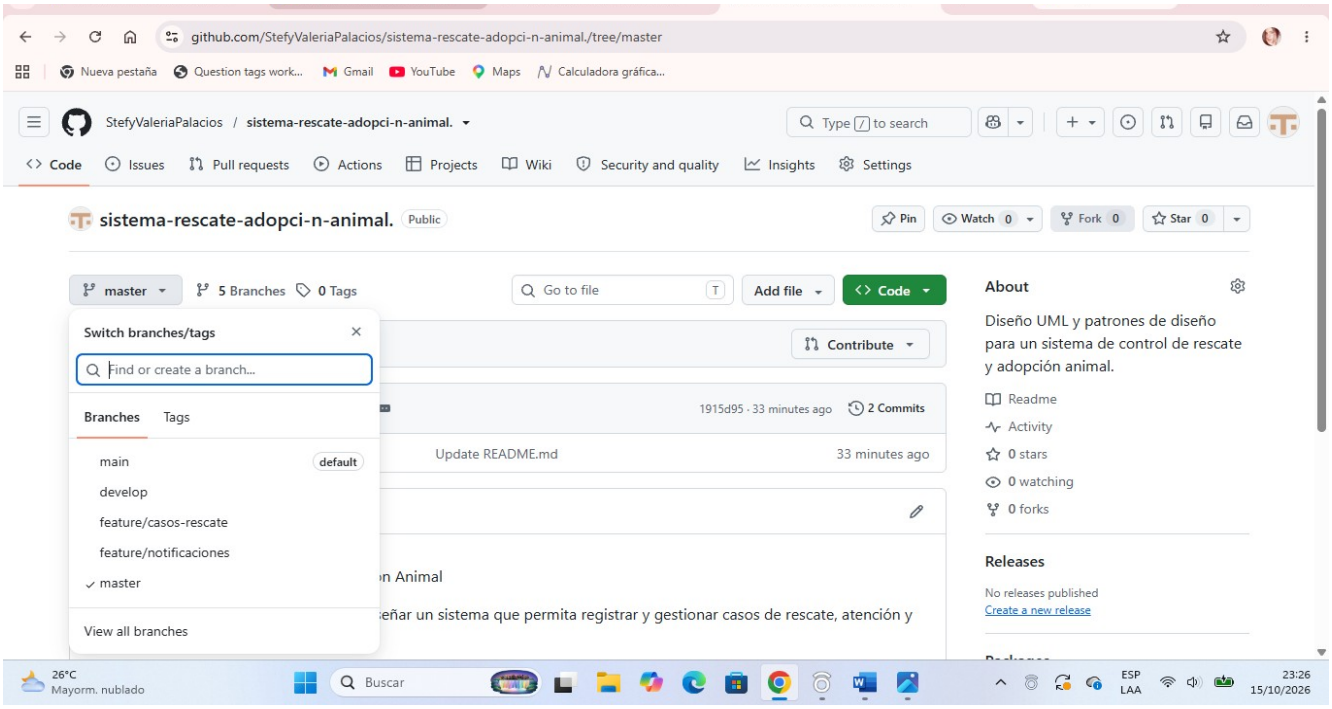
```
sistema-rescate-adopcion-animal/
  README.md
  .gitignore
  diagramas/
    casos-uso.png
    clases.png
    secuencia.png
    arquitectura-gitflow.png
  documentacion/
    informe-diseno.pdf
  src/
    modelos/
    servicios/
    controladores/
    vistas/
```



7. Configuracion de las ramas Gitflow

Se crearon las ramas main, master, develop, feature/casos-rescate y feature/notificaciones, con la finalidad de organizar el trabajo y separar el desarrollo de funcionalidades de las ramas principales.

La rama develop se utilizará para integrar los cambios de desarrollo, mientras que las ramas feature/casos-rescate y feature/notificaciones permitirán organizar las funcionalidades relacionadas con la gestión de casos y el envío de notificaciones. Las ramas main y master se conservarán hasta verificar y definir la organización final del flujo de versiones.



14. Conclusiones

El análisis de requisitos permitió definir las funciones esenciales para gestionar reportes, rescates, atención veterinaria y adopciones. La identificación de actores y responsabilidades ofrece una base clara para organizar el trabajo del refugio.

Los diagramas UML ayudan a comunicar la arquitectura antes de programar. El diagrama de casos de uso muestra las interacciones con los actores, el diagrama de clases presenta la estructura de objetos y el diagrama de secuencia explica el orden de las operaciones al registrar un rescate.

Los patrones Singleton, Observer y Factory Method abordan problemas distintos: coordinación de la gestión de casos, comunicación mediante eventos y creación de expedientes según la especie. MVC y los principios SOLID complementan la separación de responsabilidades.

Finalmente, GitHub y Gitflow proporcionan trazabilidad de los cambios y un proceso ordenado de integración. La calidad de la entrega dependerá de acompañar el diseño con un repositorio real, ramas, commits y capturas verificables.

15. Referencias

Universidad Técnica de Manabí. (2026). Guía de la Actividad #1: Estudio de caso – Sistema de Control de Rescate y Adopción Animal. Desarrollo de Sistemas Informáticos, Unidad 1.

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1. https://www.omg.org/spec/UML/2.5.1

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Atlassian. (s. f.). Gitflow Workflow. https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow

GitHub Docs. (s. f.). About pull requests. https://docs.github.com/en/pull-requests